

VCF Core: A Semantic Foundation for RDF Representations of VCF

Elias Crum^{1,2,*}, Bart Buelens², Gökhan Ertaylan² and Ruben Taelman¹

¹IDLab, Department of Electronics and Information Systems, Ghent University – imec, Belgium

²Flemish Institute for Technological Research (VITO), Mol, Belgium

Abstract

Connecting genomic observations to biomedical knowledge depends on preserving the context that makes each observation interpretable. In the Variant Call Format (VCF), overlooking version rules, field declarations, or sample-specific allele relationships can silently associate evidence with the wrong variant. Existing semantic models leave parts of this context to application-specific conventions, limiting what users can retrieve and integrate without losing context. We developed VCF Core, a vocabulary that makes this context explicit and provides version-specific representations for VCF 4.1–4.5, allowing data to be interpreted according to the specification version the file declares. This paper evaluates VCF Core’s specification coverage, robustness, and practical utility. Our reproducible workflow combines competency questions, specification-derived tests, negative controls, cross-implementation comparisons, and semantic round-tripping, with explicit expected answers and transparent reporting of passing, failing, and untested requirements. VCF Core offers two sample profiles: expanded, which gives each VCF sample value a dedicated resource, and condensed, which stores sample values in compact vectors. The specification-derived assessment demonstrates preservation in all 210 test cases in both profiles, and direct structural retrieval in all 210 expanded and 165 condensed cases, exercising 53 of 94 registered VCF specification requirements. A synthetic integration experiment further demonstrates that standard SPARQL queries can correctly join local-allele evidence to external annotations while retaining source provenance, without application-specific decoding. By making VCF interpretation context explicit through native graph relationships, VCF Core turns operations otherwise dependent on format-specific code into reusable semantic structures, providing a foundation for conversion and querying while enabling the incorporation of source observations with broader biomedical knowledge.

Keywords

Variant Call Format, RDF, Genomic data, Vocabulary, Data integration, SHACL

1. Introduction

Healthcare research and personalized medicine depend on integrating clinical observations, molecular data, and biomedical knowledge across institutions and databases. Differences in data structures, identifiers, and terminology complicate their combined use [1, 2]. Semantic Web approaches address this heterogeneity through shared identifiers, explicit relationships, and reusable vocabularies [3, 4], but meaningful integration also depends on preserving the context in which observations were recorded.

The Variant Call Format (VCF) is widely used to exchange genomic variant data [5]. Each record describes a genomic location, reference and alternate alleles, and optional annotations. Sample columns carry observations such as genotype and sequencing depth. Header declarations define fields and how their values should be interpreted [6]. The GA4GH-maintained specifications define this conserved structure across VCF 4.1–4.5, while differing in declarations and constraints [7]. VCF content fields can contain dependencies on other fields, essentially encoding meaning through the structure of the file. For example, genotype 0/2 identifies the reference allele and the second listed alternate allele, which is indexed based on the order in which these alternate alleles appear in the file, while separately an

SWAT4HCLS 2027, February 1–4, 2027, Basel, Switzerland

*Corresponding author.

✉ elias.crum@ugent.be (E. Crum); bart.buelens@vito.be (B. Buelens); gokhan.ertaylan@vito.be (G. Ertaylan); ruben.taelman@ugent.be (R. Taelman)

ORCID 0009-0005-3991-754X (E. Crum); 0000-0001-7734-3747 (B. Buelens); 0000-0001-5602-6435 (G. Ertaylan); 0000-0001-5118-256X (R. Taelman)



© 2027 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

annotation’s cardinality may depend on the corresponding header declaration. When observations are integrated with external information, those relationships must remain available.

RDF and Linked Data offer a basis for exposing this context as explicit, addressable relationships alongside links to biomedical resources [8, 9, 10]. This supports the interoperability and reuse sought by the FAIR principles [11], although wider FAIRness also depends on identifiers, metadata, access, and provenance [12].

Here, we present and assess the VCF Core Vocabulary¹ [13]. The vocabulary aims to provide VCF specification-faithful terms, rules, and modelling in a version-specific manner for VCF 4.1–4.5. This work is guided by four research questions: **RQ1**: To what extent does VCF Core preserve and represent what is defined in the VCF specification (including file and header context, allele-dependent field values, sample genotypes and phasing, version-specific constructs, etc.)? **RQ2**: Do VCF graphs generated by two different conversion methods represent the VCF specification definitions from *RQ1*, and can a source VCF file be reconstructed from generated VCF graphs (i.e., round-tripping)? **RQ3**: Can VCF graphs enable the integration of VCF data with external data while retaining the initial VCF file, record, sample, and declaration context to answer a complex question? **RQ4**: How does VCF Core compare to other existing VCF semantic models at answering nine usage-oriented information-retrieval questions from VCF data?

Through these assessments, we establish VCF Core as a version-aware representation of VCF source information and context, provide a reproducible assessment that links specification requirements to executable tests, assess the validity and faithfulness of VCF data materialized as RDF, and position and align VCF Core relative to existing semantic VCF models. Throughout, `vcfc`: denotes <https://w3id.org/vcf-core/vocab#>.

2. Related Work

Table 1 lists existing reusable semantic models and VCF–RDF conversion or query tools, with some contributions combining both roles. The Genomic Feature and Variation Ontology (GFVO), the Genome Variation Ontology (GVO), the Genotype Ontology (GENO), and HERO-Genomics model genomic or clinical concepts. Tools such as VCF2RDF, TogoVar, BioInterchange, AgroLD, and bio-vcf [14] expose VCF-derived data for integration. The GA4GH Variation Representation Specification (VRS) complements these approaches with computable variation descriptions and federated identification, using JSON Schema rather than RDF or OWL; it does not preserve source-file structure [15].

While not targeted exclusively at VCF representation, other ontologies can represent parts of the information carried by a VCF file. PROV and DCAT support provenance and dataset-level descriptions [16, 17]; SO and FALDO provide terms for sequence features and genomic locations [18, 19]; and GO provides functional context for genes and gene products [20].

3. VCF Core representation

VCF Core is a semantic vocabulary with classes, properties, declared domains and ranges, and a shallow subclass hierarchy; it carries no OWL restrictions, cardinality, disjointness, or inverse axioms. Value and cardinality constraints are expressed as SHACL shapes [34]. The vocabulary was developed iteratively from the earlier VCF-RDFizer vocabulary [35], using AI-assisted drafting [36, 37] and author review to derive terms, field definitions, and constraints from VCF 4.1–4.5 [7, 6]. Terms and relationships can be browsed in the VCF Core Vocabulary Explorer [38].

Listing 1 shows a minimal VCF example illustrating the structure and typical field values contained in a VCF file. The header declares a version, reference, contig, and three sample field definitions: genotype (GT), local alternate-allele indices (LAA), and local read depths (LAD). The FORMAT column determines the identity of items within the colon-separated sample values, which two sample columns, S1 and S2,

¹<https://ecrum19.github.io/vcf-core-vocabulary/>

Table 1

VCF-related approaches and their roles: reusable models, RDF conversion/query tools, or combinations of both.

Approach	Role and focus
GFVO [21]	OWL ontology. Harmonizes features and variation in VCF, GVF, GFF3, GTF.
GVO [22]	OWL ontology. Describes biological variation, particularly complex structural variants, rather than VCF structure and interpretation rules.
GENO [23]	Ontology. Represents genotypes, alleles, sequence alterations, and their relationships. Does not represent VCF-specific file structure.
HERO-Genomics [24]	Ontology. Integrates VCF- and Beacon-derived information across clinical and research settings, with broader scope than VCF specification coverage.
Semantic Beacons [25]	Integration mapping. Maps GA4GH Beacon responses to RDF with FALDO, SO, and GENO for federated genomic querying; not a VCF–RDF representation.
BioInterchange [21]	Converter. Emits GFVO from GFF3, GVF, and VCF; the BioInterchange VCF reader extends the GFF3 reader, so records become generic features, and it reads VCF 4.1–4.2 only.
VCF2RDF [26]	Mapping/converter. Isomorphic VCF–RDF mapping exposing file components independently with source context; not a genomic ontology.
BioHackathon <i>vcf2rdf</i> [27]	Converter. Early VCF–RDF conversion: FALDO positions, tool-specific terms.
SPARQLing genomics [28]	Converter + query platform. Generates RDF from VCF and other genomic sources and serves it for querying; the terms are tool-specific, not a VCF model.
TogoVar <i>vcf2rdf</i> [29]	Converter. Simple RDF representation of population allele-frequency VCF data for integration with TogoVar’s biomedical knowledge graphs.
AgroLD VCF2RDF [30]	Converter. Adds VCF variation to AgroLD’s plant-science knowledge graph.
VariantKG [31]	Converter + application ontology. Builds VCF-derived graphs for variant analysis and graph machine learning, not specification modelling.
VCF-RDFizer [32]	Converter. RML-based pipeline [33] that produces and verifies VCF graphs.

then supply. Unspecified VCF fields are shown as dots. Trailing sample subfields may also be dropped entirely, as S2 does on line 10.

Listing 1: Three-record, two-sample VCF example (*example.vcf*), with columns aligned for display.

```

1 ##fileformat=VCFv4.5
2 ##reference=https://example.org/reference.fa
3 ##contig=<ID=chr1,length=1000>
4 ##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
5 ##FORMAT=<ID=LAA,Number=.,Type=Integer,Description="Local ALT indices">
6 ##FORMAT=<ID=LAD,Number=LR,Type=Integer,Description="Local read depths"> ①
7 #CHROM POS ID REF ALT ② QUAL FILTER INFO FORMAT ③ S1 S2
8 chr1 1 . G A,C . . . GT:LAA:LAD 2/2:2:20,30 0/1:1:12,9
9 chr1 2 . A C,G,T . . . GT:LAA:LAD 0/3:3:15,25 0/1:1:11,7
10 chr1 3 . T C . . . GT:LAA:LAD ./.:.: 0/0

```

Figure 1 shows how VCF Core models the fields of Listing 1, in three steps marked on both. ① A `VCFFile` represents *example.vcf* and links to the `VCFHeader` (lines 1–6) and three `VCFRecord` resources (lines 8–10). The header contains `HeaderLine` resources; a `FORMATHeaderLine` is also a `FieldDefinition`, supplying a field’s identifier, type, and cardinality, as LAD does on line 6. ② Each record describes a location and the alleles there: line 8 gives the reference base “G” (REF) at position 1 of chr1 and two alternate alleles, “A” and “C” (ALT), numbered 1 and 2 after the reference’s 0; each alternate becomes an `AltAllele`. ③ The `VariantCall` term groups record values and sample observations, and a `SampleCall` describes the values within the sample column named S1 on line 7.

The `FORMAT` column splits S1’s entry on line 8, `2/2:2:20,30`, into three values. `GT=2/2` calls allele 2 (ALT[2]), “C”, on both chromosome copies, and `LAA=2` lists the alternate alleles this sample’s depths

describe, here only “C”. Line 6 (①) declares LAD with Number=LR: defined by the VCF specification as one depth for the reference allele, followed by one depth for each allele listed in LAA, not one for every ALT allele [6]. S1’s depths therefore assign 20 reads to “G” and 30 reads to “C”, whereas matching the depths to ALT by position would wrongly give 30 to “A”. VCF Core represents 30 as a `FieldValueItem` linked through `forAllele` to “C”, and, through a `FormatFieldValue`, to the header and sample.

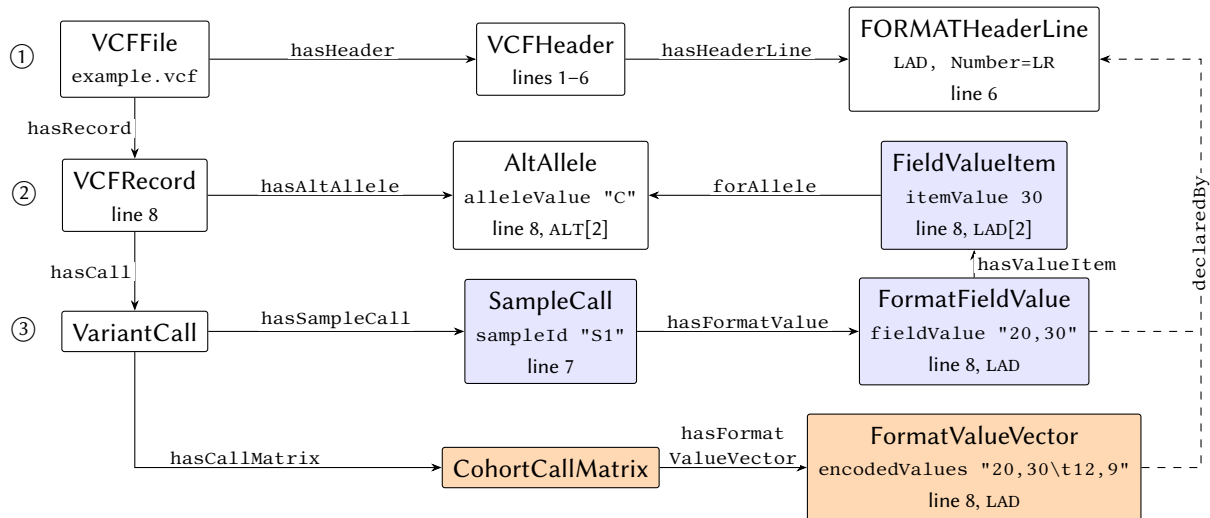


Figure 1: Selected resources for the first record (line 8) of Listing 1, walked through in steps ①–③. Blue nodes are *expanded*-profile resources, orange *condensed*. Depth 30 belongs to “C”, the sample’s local allele, not to ALT[1], “A”; bracketed positions count from 1. LAA needs no node, because `forAllele` already applies it. Dashed links show that both profiles resolve to the same `FieldDefinition`. Lines 9–10, the expanded resources for S2, other fields, allele “A”, and header lines are omitted; the condensed vector holds both samples, tab-separated.

VCF Core provides two *sample profiles* for representing sample-level observations. In the *expanded* profile, each sample value is represented as a dedicated RDF resource. In the *condensed* profile, values are stored in sample-ordered vector literals and recovered using the corresponding FORMAT definition and sample ordering (see the [GitHub documentation](#)²).

The difference is primarily one of scale for multi-sample VCF files (as used in cohort studies such as [39]), and concerns the number of RDF statements:

$$\underbrace{O(N_{\text{variants}} N_{\text{samples}} N_{\text{FORMAT}})}_{\text{(E) Expanded statements}} \quad \underbrace{O(N_{\text{variants}} N_{\text{FORMAT}})}_{\text{(C) Condensed statements}}.$$

For illustration, a VCF with 1,000 variants across 2,500 samples using the FORMAT column `GT:LAA:LAD` from above yields approximately 1.6×10^8 statements in the expanded profile and 7.9×10^4 in the condensed profile, calculated by extrapolating the statements the VCF Core materializer produces for small files (`scripts/statement-counts.py`). This reduction eases loading and storage at cohort scale, but each condensed vector still holds one value per sample, so the size of the stored values and the cost of decoding them still grow with N_{samples} . Both profiles preserve the same values; the trade-off is direct queryability, since expanded graphs support SPARQL access to individual sample values and their relationships, whereas condensed values must first be decoded. They also retain the same file, header, and record context, including version-specific registries and validation overlays selected from `fileFormat` [13].

RDF representations of `example.vcf` (Listing 1) using VCF Core, materialized using both expanded and condensed profiles, are available on [GitHub](#)³.

²<https://github.com/ecrum19/VCF-RDFizer/blob/v3.0.3/docs/sample-representation-guide.md>

³https://github.com/ecrum19/SWAT4HCLS_2027/tree/main/long_revised/examples

3.1. Assessment design

Each research question adapts an established practice: RQ1 tests specification rules stated as checkable questions [40, 41] and checks the curated inventory against an external frame of reference [42], RQ2 applies conformance testing across independent implementations [43, 44], and RQ3 and RQ4 are task-based and comparative evaluations [45].

Evaluated artifacts and reproduction. All results use VCF Core v2.1.3 [13] and VCF-RDFizer v3.0.3, with the latter’s container pinned by digest. No triple store or database is involved: each graph is an N-Triples file loaded into an in-memory rdflib 7.6.0 graph, and every SPARQL 1.1 query, including the controls, runs on rdflib’s query engine. These simple and easily reproducible methods suffice for fixtures of the tested size. Store-level performance is out of scope for this study. The *experimental repository*⁴ holds one directory per research question (RQ1/–RQ4/), each with a script for reproduction, inputs, recorded results, and an `EXPERIMENT.md` file describing the procedure and design. `long_revised/examples/` holds machine-readable versions of this paper’s listings, their materialized graphs, and the full queries. `PINS.md` lists every pinned version and digest.

4. RQ1: coverage of the VCF specification

4.1. Method

We assess correspondence between the VCF specification and VCF Core in both directions: from specification requirements to the vocabulary representation, and from the vocabulary to a curated inventory of VCF constructs.

4.1.1. VCF specification to VCF Core representation

The first assessment starts from pinned, digest-verified VCF 4.1–4.5 specification $\LaTeX$ files. Scripts read these files, extract the explicit reserved `INFO` and `FORMAT` declarations, and record a decision for every section so that no passage is skipped (RQ1/methodology/scripts/). *Requirements* were not extracted automatically but drafted from the text with AI assistance [36, 37], linked to their source passages, and reviewed by the first author (`requirements.json`; procedure in RQ1/EXPERIMENT.md). A *requirement* states a specification rule as a question with a checkable answer, following established competency-question and requirements-specification practices [40, 46, 47]. For example, R27 asks: “Can local allele indices map to global record alleles?”, the rule Listing 1 illustrates. Binary encoding, compression, indexing, producer estimation algorithms, and general invalid-input rejection are outside the scope of this assessment.

Cases and expected answers. A *case* pairs one requirement and VCF version with a *fixture*: a small VCF file constructed to expose the corresponding specification rule. The fixture is materialized as a *witness* graph using VCF Core and queried with SPARQL [48]. Each case also has an *expected answer*, derived from the specification and fixture before the query is executed, adapting SAMOD and executable ontology-testing approaches [41, 49, 50]. Where possible, fixtures and expected values are taken directly from examples in the VCF specification.

The query is evidence of coverage, not the source of the expected result. Each case passes only if the graph holds enough information to reproduce an answer defined independently from the specification and fixture. Tailoring a query to the vocabulary cannot recover information missing from the graph.

Answers are compared as multisets of string-valued rows: missing, extra, or duplicated rows fail, while requirements concerning order explicitly project the relevant ordinal. Values such as 30 and 30.0 therefore remain distinct. The fixture’s declared VCF version must also match the version assigned to the case.

⁴https://github.com/ecrum19/SWAT4HCLS_2027

Worked cases. Listing 1 illustrates the rules that requirements R27 and R21 test, in place of the registered fixtures. R27 concerns the local-allele mapping that Figure 1 follows for S1 on line 8, where depth 30 belongs to ALT allele 2, C. In S2, LAA=1 selects A instead, so the depth 9 at the same local position belongs to ALT allele 1. Listing 2 retrieves that mapping; restricted to line 8, it expects the rows (1, S1, 0, 0, G, 20), (1, S1, 1, 2, C, 30), (1, S2, 0, 0, G, 12), and (1, S2, 1, 1, A, 9) for record position, sample, local index, global index, allele, and depth. Dropping LAA would attach both second depths to ALT allele 1 and fail the case.

Listing 2: A query retrieving the local-to-global allele mapping that R27 tests.

```
PREFIX vcfc: <https://w3id.org/vcf-core/vocab#>
SELECT ?pos ?sample ?index ?global ?allele ?depth WHERE {
  ?record vcfc:pos ?pos ; vcfc:hasCall/vcfc:hasSampleCall ?call .
  ?call vcfc:sampleId ?sample ; vcfc:hasFormatValue ?field .
  ?field vcfc:declaredBy/vcfc:fieldId "LAD" ; vcfc:hasValueItem ?item .
  ?item vcfc:valueIndex ?index ; vcfc:itemValue ?depth ; vcfc:forAllele ?a .
  ?a vcfc:alleleIndex ?global ; vcfc:alleleValue ?allele .
} ORDER BY ?pos ?sample ?index
```

R21 concerns missingness, which line 10 of Listing 1 exhibits. S1 contains ./.:.:. , which explicitly reports a missing LAD. S2 contains 0/0, which drops the trailing LAA and LAD subfields altogether. The two remain distinguishable, so the expected results for line 10 are (S1, ".") and (S2, unbound). The expanded query follows each sample's LAD field, using OPTIONAL to retain a row when that field is absent. The cases for both requirements above use their own fixtures, queries, and expected results, and are documented in [RQ1/methodology/](#) in the experimental repository.

Retrieval, controls, and scoring. Each case runs in both sample profiles on two *axes*. *Preservation* permits decoding values from literals, whereas *structure* requires graph relationships and prohibits the string-decomposition functions REPLACE, SUBSTR, STRBEFORE, and STRAFTER. An empty-graph control rejects queries that return the expected answer without data. A predicate-deletion control requires the answer to change after deleting at least one referenced property; it establishes dependence on some vocabulary data, rather than on every property mentioned. Requirements are scored per applicable version, profile, and axis: demonstrated when all cases pass, partial when some pass, not demonstrated when all fail, and unassessed when no case exists. Only demonstrated requirements enter the numerator of coverage; all applicable requirements, including unassessed ones, remain in the denominator with equal weight.

The VCF Core vocabulary repository's Python materializer⁵ builds witnesses from fixtures. The materializer, vocabulary, and queries share an author, so a common misunderstanding remains possible despite separately authored answers and controls: the controls establish dependence on graph data, not that a question or expected answer correctly interprets the specification. Test selection also favours rules with defensible examples. RQ2 tests whether the same questions expose representation differences in a separate implementation, but does not provide independent validation of the questions. Separately, reserved INFO/FORMAT Number/Type declarations are extracted from the specification texts and compared with the corresponding version registries. This tests explicit declarations, independently of prose interpretation.

4.1.2. Curated inventory

The second assessment links 104 VCF 4.5 logical constructs to vocabulary terms and supporting evidence. The inventory classifies whether information is preserved, structurally retrievable, and enforced by a validation rule. Representation is full when both preservation and structure are supported, partial

⁵https://github.com/ecrum19/vcf-core-vocabulary/blob/v2.1.3/scripts/vcf_examples.py

when only preservation is supported, and absent otherwise. Enforcement is recorded separately and does not contribute to the representation score. The two assessments count different things and are not combined: the first inspects specification-derived requirements in both profiles, whereas the inventory records curated judgments about VCF constructs in the expanded profile only.

4.2. Results

Declarations and assessment corpus. All 333 explicit reserved Number/Type declarations extracted from VCF 4.1–4.5 match the version registries, with no missing or differing row. VCF 4.5’s registry contains 122 reserved definitions: 51 INFO and 71 FORMAT, including three parameterized base-modification families. These are different populations: the comparison includes only explicit, comparable declarations, while older specifications also define reserved keys in prose without complete Number/Type information. Agreement is therefore weighted towards VCF 4.3–4.5 and does not settle declaration–prose inconsistencies, such as CNL/CNP declaring Number=G while describing copy-number ordering.

The VCF specification was decomposed into **94 requirements**, **38 fixtures**, **210 cases**, and **79 distinct queries**, linked to specification passages and expected answers. Nine fixtures reproduce published specification examples exactly; the others are minimal files instantiating stated rules. Interpretation statements also vary in specificity: 56 are requirement-specific, 29 generic, and nine explicitly unassessed. During our review we identified inconsistent specification examples, including three issues (such as [hts-specs#869](https://github.com/samtools/hts-specs/issues/869)⁶) reported to the maintainers and one previously reported issue, documented in the experimental repository. Where an interpretation could be established, affected requirements were scored; unresolved cases remained unassessed and counted against coverage.

Executed coverage. The 210 cases exercise **53 of 94 requirements** in at least one applicable version. All 210 preservation tests passed in each profile, as did all 210 expanded-structure tests. Condensed structure passed 165 and failed 45 tests, all concerning sample-level FORMAT access held in vectors rather than individual resources. Every passing case rejected the empty-graph control and changed the answer under at least one referenced-predicate deletion. The worked cases follow this pattern: R27 and R21 both pass preservation in both profiles, and structure only in the expanded profile. Figure 2 aggregates these outcomes by requirement and applicable version.

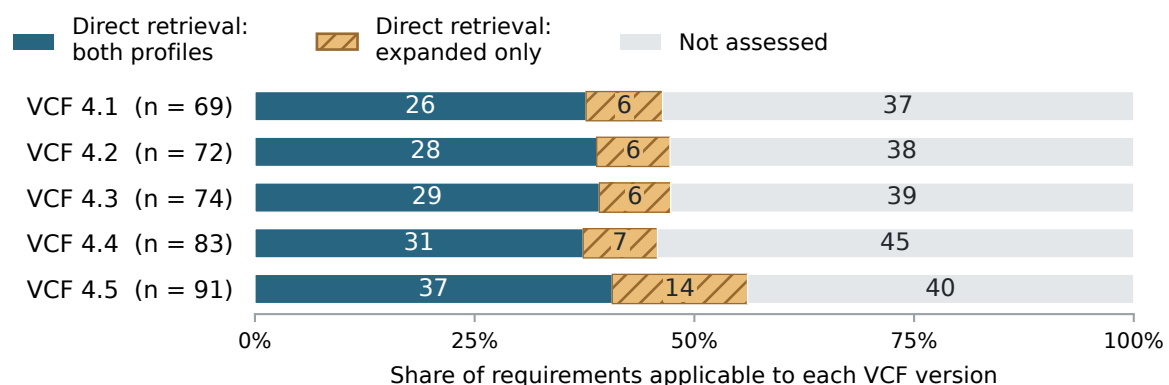


Figure 2: Coverage of registered requirements by VCF version; the legend names the three categories. Colored segments pass preservation in both profiles. Widths show percentages of applicable requirements and labels provide counts. Requirements may apply to more than one version.

For VCF 4.5, 51 of 91 requirements (56.0%) have demonstrated preservation in both profiles; 37 also support direct retrieval in condensed data, while 40 remain unassessed. Of the 40 unassessed

⁶<https://github.com/samtools/hts-specs/issues/869>

requirements, 37 await fixtures and queries, one requires further interpretation, and two require coverage of all reserved INFO and FORMAT keys. Most missing coverage therefore reflects absent tests rather than failures: the tested structural failures concern only the condensed representation, and their expanded counterparts pass.

Curated inventory. All **104** inventoried VCF 4.5 logical constructs were classified as preserved and structured. **87** of these constructs have a validation rule (i.e., a SHACL shape or a decoded check) testing a stated requirement; the inventory and supporting evidence are in [RQ1/vcf45-inventory/](#).

5. RQ2: cross-implementation consistency and round-tripping

5.1. Method

RQ2 examines whether VCF Core can support a second conversion strategy and whether structured RDF retains enough information to recover the source observations. We re-execute all RQ1 cases against witness graphs generated by VCF-RDFizer v3.0.3 [32], keeping fixtures, queries, and expected answers unchanged. This converter executes RML mappings [33] with RMLStreamer [51], whereas the VCF Core materializer constructs triples via a Python script. The programs share an author, so agreement cannot rule out a shared interpretation error. The comparison can nevertheless reveal implementation-specific omissions or incorrect associations, and the round-trip separately checks recovered values against the source VCF. Every fixture is converted in both profiles, and both producers' graphs are generated during the run. For each case, axis, and profile, we record whether both pass, either producer alone passes, or both fail. We also compare the two profiles case by case. Because producers mint different resource bases, fragment-bearing IRIs are compared by fragment alone; a wrong namespace with the correct fragment could therefore escape detection.

Semantic round-trip. A complementary check, run on expanded graphs only, reconstructs the seven fixed columns, sample genotypes, header lines, INFO entries, and non-GT FORMAT values from structured RDF properties and compares them with the source. The preserved-source properties `infoRaw`, `sampleDataRaw`, and `genotypeString` are not used. Deliberate alterations or deletions of genotype, field, and header data verify that mismatches are detected. A leading phase indicator is normalized because VCF 4.4–4.5 permit equivalent forms such as `| 0 | 1` and `0 | 1`, but allele order, missing calls, ploidy, and intervening phase indicators must still match exactly. A condensed round-trip was not run: the round-trip check has no decoder for the encoded sample vectors yet, although RQ1's preservation queries show that those values can be decoded. Only sample-level values would differ, since both profiles agree on every query above the sample level (Section 5.2).

5.2. Results

Re-executing the 210 cases on two axes and two profiles gives **840 checks: 693 pass for both producers, 45 fail for both, and 102 differ** across 15 requirements. Every disagreement is a materializer pass and converter failure; none shows the inverse. The 45 shared failures are condensed-structure queries requiring per-sample resources. Of the 53 exercised requirements, 38 agree on every check and 32 pass on every check. Inspection of each disagreement identifies the causes in Table 2. No observed divergence requires a representational mechanism absent from the pinned vocabulary; every missing property is already declared.

An earlier comparison also exposed a silent local-allele error, missed by the converter's existing tests: the converter associated `Number=LA` and `Number=LR` values with record alleles positionally instead of resolving the sample's local subset. Under `LAA=2, 4`, depths `20, 30, 10` belong to global alleles `0, 2, and 4`, but were attached to `0, 1, and 2`. That defect and eight further divergences were repaired [32]. In the evaluated release, all local-allele requirements pass for both producers on expanded-profile checks.

Table 2

Causes of disagreement across 15 requirements.

Cause	Reqs.	Finding
Unemitted properties	12	Twenty declared properties absent, including accessors for gVCF blocks, breakend mates, and allele indices.
Incomplete decomposition	1	Tandem-repeat summary emitted without per-allele components.
Permitted datatype difference	1	QUAL uses <code>vcfc:VCFFloat</code> or <code>xsd:decimal</code> ; both satisfy the shape but differ under <code>DATATYPE()</code> .
Encoding convention	1	Different handling of an empty local-allele list (R27).

Semantic round-trip and profile comparison. Across 38 fixtures, reconstruction from structured properties recovers the seven fixed columns of all **109 records**, all **151 sample genotypes**, all **431 header lines**, and **400 of 401** INFO entries and non-GT FORMAT subfields. The exception is an empty LAA list: the materializer writes an empty value, while the converter emits the field without a value triple. This loses the distinction between a present, empty list, meaning reference-only local alleles, and a field without a value. The round-trip thus provides a complementary check on R27. The associated LAD item retains the `forAllele` link, so the error does not remove that depth association.

The sample profiles give the same outcome for both producers on **352 of 420** case-and-axis comparisons. All 68 disagreements concern FORMAT values or genotypes within samples, spanning 15 requirements. Questions about files, headers, records, and alleles agree throughout. The profiles therefore agree above the sample level; below it, condensed values must be decoded first.

6. RQ3: integration with external data

6.1. Method

RQ3 asks which externally flagged alterations have sufficient sample depth and which source observations support the answer. The experiment has three inputs, all in `RQ3/inputs/` of the experimental repository. The first is a VCF fixture, `local-alleles-v4.5.vcf`, of four synthetic sites for one sample column, each site written twice: with local-allele (LAA, LAD, LPL) and with global (AD, PL) fields. The second, `annotations.ttl`, is a separately authored snapshot of five invented, flagged alterations that shares no identifiers with the fixture. The third is the query, `local-allele-evidence.rq`, excerpted in Listing 3. VCF-RDFizer v3.0.3 [32] converts the fixture in the expanded profile, and the query runs over that graph together with the snapshot, joining on `contig`, `position`, `reference allele`, and `alternate allele`, then retrieving supporting depth through `forAllele`. This assumes the same reference assembly, unchecked against the VCF reference declaration, and identical as-written alleles without normalization or VRS identification. The sample identifies a VCF column, not a patient.

The query keeps only depths of at least 20 (`LAD >= 20`), distinguishing correct allele-depth joins from positional misreading while retaining source context. Expected rows, authored before execution, are compared across eight answer fields, with a ninth, the source file, checked against the converted input path. Each of the three excluded alterations is checked individually. The example concerns only the `Number=LR` field LAD. The snapshot’s annotations have no biological meaning and are stored with the experiment so that every run returns the same answer.

6.2. Results

On the fixture, the query returns the two expected alterations and retains the correct file, record, sample, and FORMAT declaration context.

To explain the result without a further listing, Table 3 and Figure 1 use Listing 1 rather than the experimental fixture. In Listing 1, lines 8–9 are simplified versions of the fixture’s first two local-allele records, keeping the positions, reference alleles, and the local alleles and depths that the joins use, with

Listing 3: Graph-pattern excerpt from the WHERE clause of local-allele-evidence.rq. Prefix declarations, SELECT, filtering, and provenance retrieval are omitted.

```
# 1. the flagged alteration, from the external annotation snapshot
?annotation a ex:FlaggedAlteration ; ex:contig ?contig ; ex:position ?position ;
    ex:referenceAllele ?ref ; ex:alternateAllele ?alt .
# 2. the same alteration as written in the VCF file
?record vcfc:chrom ?contig ; vcfc:pos ?position ; vcfc:ref ?ref ;
    vcfc:hasCall ?call ; vcfc:hasAltAllele ?altAllele .
?altAllele vcfc:alleleValue ?alt .
# 3. that sample's LAD value, and the declaration giving LAD's cardinality
?call vcfc:hasSampleCall/vcfc:hasFormatValue ?field .
?field vcfc:declaredBy [ vcfc:fieldId "LAD" ; vcfc:fieldNumber ?number ] ;
    vcfc:hasValueItem ?item .
# 4. the depth for THIS allele, via the sample's own local allele set
?item vcfc:forAllele ?altAllele ; vcfc:itemValue ?depth .
```

S1 representing the fixture's sample. ① For chr1:1 G>C, the four location and allele values listed in the annotation (long_revised/examples/annotations.ttl) identify the record's "C" allele, and forAllele leads to depth 30 (Figure 1). ② For chr1:1 G>A, the same location matches, but no LAD value of S1 is linked to "A", so the query excludes the alteration. S2 is not considered here.

Table 3

Two readings of S1's allele depths on lines 8–9 of Listing 1: through forAllele, as Number=LR requires, or by position in ALT. Each cell gives the allele that receives the second depth and, in parentheses, the flagged alteration that depth supports. Both depths pass the threshold of 20, so only the reading decides the answer.

Record	LAA	LAD	Through forAllele	By ALT position (incorrect)
line 8, chr1:1	2	20,30	30 → C (G>C) ①	30 → A (G>A) ②
line 9, chr1:2	3	15,25	25 → T (A>T)	25 → C (A>C)

The incorrect interpretation still returns two plausible rows with provenance. This counterexample was defined from the fixture and specification before execution, and illustrates the rule, rather than measuring a converter error.

7. RQ4: comparison with existing semantic models

7.1. Method

Published artifacts for GFVO, HERO-Genomics, GVO, and VCF2RDF were inspected against nine retrieval criteria (Table 4) [21, 24, 22, 26]. Verdicts are **E**, explicitly represented; **C**, representable with additional conventions; and **S**, outside stated scope. The criteria concern VCF interpretation context, and **S** denotes a difference of purpose rather than a defect. F7 covers only per-allele lists: Number=G values index possible genotypes, and although VCF Core records each item's genotype index, decoding it into alleles relies on the specification's ordering and is untested (requirement R37). Artifact digests identify the inspected versions. VCF2RDF's terms were inspected, while converter behaviour was taken from the VCF2RDF paper rather than execution. GA4GH VRS is not scored because it addresses computable variation identity rather than source-file representation [15]. This assessment measures documented support, not the capabilities of extensions or applications built around these models. The verdicts reflect one inspection, without an inter-rater assessment.

7.2. Results

Table 4 summarizes the inspected artifacts. HERO-Genomics and GVO retain INFO as one literal through `hero:vcfInfo` and `gvo:info`, requiring consumers to parse the text and supply interpretation rules [24, 22]. GFVO directly models field meanings such as `AlleleCount` and `MappingQuality`, but leaves the declaration mechanism out of scope [21]. VCF2RDF publishes declaration terms, including `Number`, `Type`, and `Description`, but no allele-index term, so allele association requires additional conventions [26]. GVO’s predominantly S verdicts reflect the GVO focus on classifying biological variation rather than representing source-file relationships.

Table 4

Support in inspected artifacts: E, explicitly represented; C, representable with additional conventions; S, outside stated scope. These are inspection judgments about VCF interpretation context.

Can a user...	VCF Core	GFVO	HERO	GVO	VCF2RDF
F1 trace a record to the source file	E	E	E	S	C
F2 read declaration ID, Number, and Type	E	S	S	S	E
F3 read the declared VCF version	E	S	S	S	C
F4 address ALT allele <i>n</i> by index	E	C	C	S	C
F5 separate ploidy, allele calls, and phasing	E	C	C	S	C
F6 get a (sample, field) value unsplit	E	C	C	S	C
F7 bind a <code>Number=A/R</code> item to the allele	E	C	C	S	C
F8 distinguish explicit . from unstated	E	S	C	C	C
F9 identify the sample encoding	E	S	S	S	S

VCF Core’s distinguishing role is the explicit representation of source interpretation context: declarations, version rules, allele indices, and sample scope. These findings concern the published artifacts and do not show that other models cannot support the same information through extensions. Separately, VCF Core provides **123 curated alignments**⁷, recorded in SSSOM with SKOS predicates: 57 exact, 40 close, 16 related, and 10 narrow matches [52, 53]. These graded term mappings support connections to other models without establishing identity between instances.

8. Discussion

Assessing VCF Core’s coverage of VCF meaning. The results show strong representation but incomplete demonstrated coverage: all tested expanded-profile requirements pass, many remain unassessed (Figure 2), and condensed structure scores are lower because sample values there need decoding.

Extending demonstrated coverage. The clearest next step is to broaden testing of structural-variant fields, allele-dependent values, version-specific rules, and constructs that currently lack enforcement evidence. New fixtures should isolate individual rules, derive expected answers from the specification, and exercise all applicable versions [41, 49]. Because the present tests were often guided by clear specification examples, the demonstrated subset should not be treated as representative of all VCF complexity. Turning specification rules into executable tests also exposed inconsistencies and ambiguities in examples and declarations. Such tests therefore provide useful feedback for specification maintenance by making an interpretation explicit and reviewable, as seen in other conformance-testing settings [43, 44]. Extending the fixture set with additional production files, counterexamples, and clarified requirements would strengthen vocabulary coverage and grounding in the VCF specification [54].

A reusable assessment for conversion engines. Because the assessment is defined against the shared vocabulary, RQ2 shows that it can be reused across separate conversion strategies, evaluating each

⁷<https://github.com/ecrum19/vcf-core-vocabulary/tree/v2.1.3/mappings>

converter against the same requirements before comparing performance or graph size. The assessment detected both incorrect value-to-allele associations and remaining representation mismatches, showing why structural checks should be complemented by tests of recovered values and their relationships. The current results support this reuse across two implementations, although their shared authorship means independent reproduction remains necessary future work.

Integrating observations with external knowledge. RQ3 uses local alleles to show why VCF interpretation context matters when observations are linked to external knowledge. VCF Core makes the sample-specific allele association explicit through `forAllele`, which the specification instead recommends handling in external libraries [6]. The example shows that this interpretation can be resolved during conversion and reused by downstream queries, rather than decoded repeatedly. However, the demonstration is synthetic and assumes a shared assembly and identical allele representation; realistic biomedical integration and normalization should therefore be assessed in future work.

VCF Core among existing models. Because the comparison criteria in Table 4 reflect the problems VCF Core was designed to solve, RQ4’s results should be read within that scope rather than as a general ranking of genomic ontologies. In practice, richer workflows will require these models to be combined: VCF Core can preserve source observations, quality values, sample context, and provenance, while normalized variants are linked to models such as GA4GH VRS [15] and clinical entities are connected through models such as HERO-Genomics [24], with Med2RDF and GVAR as further candidates [55, 56]. The published mappings express graded relationships between terms rather than asserting instance identity; review by those communities would strengthen both the alignments and the comparison [3].

Future work and independent evaluation. Future work should test VCF Core in realistic biomedical workflows, such as linking pharmacogenomic interpretations to supporting observations [57, 58], rare-disease candidates to phenotype knowledge [59, 2], or tumour observations to therapeutic evidence [60]. These scenarios require biomedical models, reviewed instance mappings, and explicit sample–specimen–patient relationships, with success measured by answer correctness and traceability.

Larger cohort studies should also measure conversion, storage, loading, and query costs alongside semantic correctness. The current round-trip and integration evaluations cover only the expanded profile, but approaches such as SciSPARQL querying of condensed vectors should be investigated [61].

9. Conclusion

VCF Core provides a shared semantic foundation for representing VCF observations together with the context needed to interpret them. Our assessments demonstrate structural retrieval for the tested requirements in the expanded profile and agreement between two separately implemented producers on 738 of 840 checks, with the remaining differences mostly traced to properties the converter does not yet emit.

By making field declarations, allele relationships, and sample context explicit, VCF Core enables converters and downstream applications to reuse the same interpretation rules while linking source observations to broader biomedical models.

Declaration on Generative AI

Claude Opus 5 [36] and GPT-6 Astra [37] assisted with vocabulary development, drafting the RQ1 requirements, assessment scripts, initial SPARQL query drafts, and proofreading manuscript figure captions and text. The authors reviewed all AI outputs and take full responsibility for the manuscript, experiments, tools, and repositories.

References

- [1] C. Gaudet-Blavignac, J. L. Raisaro, V. Touré, S. Österle, K. Crameri, C. Lovis, A national, semantic-driven, three-pillar strategy to enable health data secondary usage interoperability for research within the Swiss Personalized Health Network: Methodological study, *JMIR Medical Informatics* 9 (2021) e27591. URL: <https://medinform.jmir.org/2021/6/e27591/>. doi:10.2196/27591.
- [2] T. E. Putman, K. Schaper, N. Matentzoglou, V. P. Rubinetti, F. S. Alquaddoomi, C. Cox, J. H. Caufield, G. Elsarboukh, S. Gehrke, H. Hegde, J. T. Reese, I. Braun, R. M. Bruskiewich, L. Cappelletti, S. Carbon, A. R. Caron, L. E. Chan, C. G. Chute, K. G. Cortes, V. De Souza, T. Fontana, N. L. Harris, E. L. Hartley, E. Hurwitz, J. O. B. Jacobsen, M. Krishnamurthy, B. J. Laraway, J. A. McLaughlin, J. A. McMurry, S. A. T. Moxon, K. R. Mullen, S. T. O'Neil, K. A. Shefchek, R. Stefancsik, S. Toro, N. A. Vasilevsky, R. L. Walls, P. L. Whetzel, D. Osumi-Sutherland, D. Smedley, P. N. Robinson, C. J. Mungall, M. A. Haendel, M. C. Munoz-Torres, The Monarch Initiative in 2024: an analytic platform integrating phenotypes, genes and diseases across species, *Nucleic Acids Research* 52 (2024) D938–D949. URL: <https://academic.oup.com/nar/article/52/D1/D938/7449493>. doi:10.1093/nar/gkad1082.
- [3] B. Smith, M. Ashburner, C. Rosse, J. Bard, W. Bug, W. Ceusters, L. J. Goldberg, K. Eilbeck, A. Ireland, C. J. Mungall, The OBI Consortium, N. Leontis, P. Rocca-Serra, A. Ruttenberg, S.-A. Sansone, R. H. Scheuermann, N. Shah, P. L. Whetzel, S. Lewis, The OBO Foundry: coordinated evolution of ontologies to support biomedical data integration, *Nature Biotechnology* 25 (2007) 1251–1255. URL: <https://www.nature.com/articles/nbt1346>. doi:10.1038/nbt1346.
- [4] F. Belleau, M.-A. Nolin, N. Tourigny, P. Rigault, J. Morissette, Bio2RDF: Towards a mashup to build bioinformatics knowledge systems, *Journal of Biomedical Informatics* 41 (2008) 706–716. URL: <https://www.sciencedirect.com/science/article/pii/S1532046408000415>. doi:10.1016/j.jbi.2008.03.004.
- [5] P. Danecek, A. Auton, G. Abecasis, C. A. Albers, E. Banks, M. A. DePristo, R. E. Handsaker, G. Lunter, G. T. Marth, S. T. Sherry, G. McVean, R. Durbin, 1000 Genomes Project Analysis Group, The variant call format and VCFtools, *Bioinformatics* 27 (2011) 2156–2158. URL: <https://academic.oup.com/bioinformatics/article/27/15/2156/402296>. doi:10.1093/bioinformatics/btr330.
- [6] SAM/BAM Format Specification Working Group, The Variant Call Format Specification: VCFv4.5 and BCFv2.2, 2026. URL: <https://samtools.github.io/hts-specs/VCFv4.5.pdf>, 25 February 2026 printing, version e821e4f; accessed 6 September 2026.
- [7] SAM/BAM Format Specification Working Group, HTS format specifications: VCF specifications and earlier versions, Official specification index, 2026. URL: <https://samtools.github.io/hts-specs/>, accessed 9 September 2026.
- [8] S. Decker, S. Melnik, F. van Harmelen, D. Fensel, M. Klein, J. Broekstra, M. Erdmann, I. Horrocks, The Semantic Web: the roles of XML and RDF, *IEEE Internet Computing* 4 (2000) 63–73. URL: <https://ieeexplore.ieee.org/document/877487/>. doi:10.1109/4236.877487.
- [9] C. Bizer, T. Heath, T. Berners-Lee, Linked Data—the story so far, *International Journal on Semantic Web and Information Systems* 5 (2009) 1–22. doi:10.4018/jswis.2009081901.
- [10] I. M. Lubin, N. Aziz, L. J. Babb, D. Ballinger, H. Bisht, D. M. Church, S. Cordes, K. Eilbeck, F. Hyland, L. Kalman, M. Landrum, E. R. Lockhart, D. Maglott, G. Marth, J. D. Pfeifer, H. L. Rehm, S. Roy, Z. Tezak, R. Truty, M. Ullman-Cullere, K. V. Voelkerding, E. A. Worthey, A. W. Zaranek, J. M. Zook, Principles and recommendations for standardizing the use of the next-generation sequencing variant file in clinical settings, *The Journal of Molecular Diagnostics* 19 (2017) 417–426. doi:10.1016/j.jmoldx.2016.12.001.

- [11] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, G. Appleton, M. Axton, A. Baak, N. Blomberg, J.-W. Boiten, L. B. da Silva Santos, P. E. Bourne, J. Bouwman, A. J. Brookes, T. Clark, M. Crosas, I. Dillo, O. Dumon, S. Edmunds, C. T. Evelo, R. Finkers, A. Gonzalez-Beltran, A. J. G. Gray, P. Groth, C. Goble, J. S. Grethe, J. Heringa, P. A. C. 't Hoen, R. Hooft, T. Kuhn, R. Kok, J. Kok, S. J. Lusher, M. E. Martone, A. Mons, A. L. Packer, B. Persson, P. Rocca-Serra, M. Roos, R. van Schaik, S.-A. Sansone, E. Schultes, T. Sengstag, T. Slater, G. Strawn, M. A. Swertz, M. Thompson, J. van der Lei, E. van Mulligen, J. Velterop, A. Waagmeester, P. Wittenburg, K. Wolstencroft, J. Zhao, B. Mons, The FAIR guiding principles for scientific data management and stewardship, *Scientific Data* 3 (2016) 160018. URL: <https://www.nature.com/articles/sdata201618>. doi:10.1038/sdata.2016.18.
- [12] C. Bahim, C. Casorrán-Amilburu, M. Dekkers, E. Herczog, N. Loozen, K. Repanas, K. Russell, S. Stall, The FAIR Data Maturity Model: An approach to harmonise FAIR assessments, *Data Science Journal* 19 (2020) 41. URL: <https://datascience.codata.org/articles/dsj-2020-041>. doi:10.5334/dsj-2020-041.
- [13] E. Crum, VCF Core Vocabulary, Version 2.1.3. GitHub repository, 2026. URL: <https://github.com/ecrum19/vcf-core-vocabulary/tree/8a5f23a>, version 2.1.3 with the SSSOM alignment set; source snapshot of 22 September 2026.
- [14] E. Garrison, Z. N. Kronenberg, E. T. Dawson, B. S. Pedersen, P. Prins, A spectrum of free software tools for processing the VCF variant call format: vcflib, bio-vcf, cyvcf2, hts-nim and slivar, *PLOS Computational Biology* 18 (2022) e1009123. doi:10.1371/journal.pcbi.1009123.
- [15] A. H. Wagner, L. Babb, G. Alterovitz, M. Baudis, M. Brush, D. L. Cameron, M. Cline, M. Griffith, O. L. Griffith, S. E. Hunt, D. Kreda, J. M. Lee, S. Li, J. Lopez, E. Moyer, T. Nelson, R. Y. Patel, K. Riehle, P. N. Robinson, S. Rynearson, H. Schuilenburg, K. Tsukanov, B. Walsh, M. Konopko, H. L. Rehm, A. D. Yates, R. R. Freimuth, R. K. Hart, The GA4GH Variation Representation Specification: A computational framework for variation representation and federated identification, *Cell Genomics* 1 (2021) 100027. doi:10.1016/j.xgen.2021.100027.
- [16] L. Moreau, P. Groth, J. Cheney, T. Lebo, S. Miles, The rationale of PROV, *Web Semantics* 35 (2015) 235–257. doi:10.1016/j.websem.2015.04.001.
- [17] R. Albertoni, D. Browning, S. J. D. Cox, A. González-Beltrán, A. Perego, P. Winstanley, The W3C Data Catalog Vocabulary, version 2: Rationale, design principles, and uptake, *Data Intelligence* 6 (2024) 457–487. doi:10.1162/dint_a_00241.
- [18] K. Eilbeck, S. E. Lewis, C. J. Mungall, M. Yandell, L. Stein, R. Durbin, M. Ashburner, The Sequence Ontology: a tool for the unification of genome annotations, *Genome Biology* 6 (2005) R44. URL: <https://link.springer.com/article/10.1186/gb-2005-6-5-r44>. doi:10.1186/gb-2005-6-5-r44.
- [19] J. T. Bolleman, C. J. Mungall, F. Strozzi, J. Baran, M. Dumontier, R. J. P. Bonnal, R. Buels, R. Hoehndorf, T. Fujisawa, T. Katayama, P. J. A. Cock, FALDO: a semantic standard for describing the location of nucleotide and protein feature annotation, *Journal of Biomedical Semantics* 7 (2016) 39. URL: <https://link.springer.com/article/10.1186/s13326-016-0067-z>. doi:10.1186/s13326-016-0067-z.
- [20] The Gene Ontology Consortium, The Gene Ontology knowledgebase in 2026, *Nucleic Acids Research* 54 (2026) D1779–D1792. doi:10.1093/nar/gkaf1292.
- [21] J. Baran, B. S. B. Durgahee, K. Eilbeck, E. Antezana, R. Hoehndorf, M. Dumontier, GFVO: the Genomic Feature and Variation Ontology, *PeerJ* 3 (2015) e933. URL: <https://peerj.com/articles/933/>. doi:10.7717/peerj.933.
- [22] S. Kawashima, T. Fujisawa, T. Katayama, Genome Variation Ontology for annotation of complex structural variations, in: *Proceedings of the 14th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2023)*, volume 3415 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2023, pp. 126–127. URL: <https://ceur-ws.org/Vol-3415/paper-22.pdf>.

- [23] K. A. Shefchek, N. L. Harris, M. Gargano, N. Matentzoglou, D. Unni, M. Brush, D. Keith, T. Conlin, N. Vasilevsky, X. A. Zhang, J. P. Balhoff, L. Babb, S. M. Bello, H. Blau, Y. Bradford, S. Carbon, L. Carmody, L. E. Chan, V. Cipriani, A. Cuzick, M. Della Rocca, N. Dunn, S. Essaid, P. Fey, C. Grove, J.-P. Gouridine, A. Hamosh, M. Harris, I. Helbig, M. Hoatlin, M. Joachimiak, S. Jupp, K. B. Lett, S. E. Lewis, C. McNamara, Z. M. Pendlington, C. Pilgrim, T. Putman, V. Ravanmehr, J. Reese, E. Riggs, S. Robb, P. Roncaglia, J. Seager, E. Segerdell, M. Similuk, A. L. Storm, C. Thaxon, A. Thessen, J. O. B. Jacobsen, J. A. McMurry, T. Groza, S. Köhler, D. Smedley, P. N. Robinson, C. J. Mungall, M. A. Haendel, M. C. Munoz-Torres, D. Osumi-Sutherland, The Monarch Initiative in 2019: An integrative data and analytic platform connecting phenotypes to genotypes across species, *Nucleic Acids Research* 48 (2020) D704–D715. doi:10.1093/nar/gkz997.
- [24] M. Cazzaro, I. G. Gut, L. Menotti, M. Rueda, G. Silvello, HERO-Genomics: An ontology for integration and access of multicenter genomic data, in: *Proceedings of the 16th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, volume 4196 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025, pp. 33–42. URL: https://ceur-ws.org/Vol-4196/paper_17.pdf.
- [25] A. Bodrug-Schepers, H. Chabane, G. Montoya, P. Serrano-Alvarado, R. Redon, A. Gaignard, Semantic Beacons: a framework to support federated querying over genomic variants and public knowledge graphs, in: *Proceedings of the 16th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, volume 4196 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025, pp. 21–32. URL: https://ceur-ws.org/Vol-4196/paper_16.pdf.
- [26] E. D. S. Penha, E. Iriabho, A. Dussaq, D. M. de Oliveira, J. S. Almeida, Isomorphic semantic mapping of variant call format (VCF2RDF), *Bioinformatics* 33 (2017) 547–548. URL: <https://academic.oup.com/bioinformatics/article/33/4/547/2593587>. doi:10.1093/bioinformatics/btw652.
- [27] T. Katayama, S. Kawashima, G. Micklem, S. Kawano, J.-D. Kim, S. Kocbek, S. Okamoto, Y. Wang, H. Wu, A. Yamaguchi, et al., BioHackathon series in 2013 and 2014: improvements of semantic interoperability in life science data and services, *F1000Research* 8 (2019) 1677. doi:10.12688/f1000research.18238.1.
- [28] R. Janssen, UMC Utrecht Genetics, SPARQLing genomics, Version 0.99.11. Open-source software, 2020. URL: <https://gitlab.com/roelj/sparqling-genomics>, ontologies, RDF generators and a query interface for genomics; GPL-3.0. Released 19 June 2020.
- [29] N. Mitsuhashi, L. Toyo-oka, T. Katayama, M. Kawashima, S. Kawashima, K. Miyazaki, T. Takagi, TogoVar: A comprehensive Japanese genetic variation database, *Human Genome Variation* 9 (2022) 44. doi:10.1038/s41439-022-00222-9.
- [30] P. Larmande, B. Pittolat, N. Tando, Y. Pomie, B. G. Happi Happi, V. Guignon, M. Ruiz, AgroLD: a knowledge graph for the plant sciences, *BMC Genomic Data* 26 (2025) 73. doi:10.1186/s12863-025-01359-6.
- [31] S. Prasanna, A. Kumar, D. Rao, E. J. Simoes, P. Rao, A scalable tool for analyzing genomic variants of humans using knowledge graphs and graph machine learning, *Frontiers in Big Data* 7 (2025) 1466391. doi:10.3389/fdata.2024.1466391.
- [32] VCF-RDFizer maintainers, VCF-RDFizer, Version 3.0.3. Open-source software, 2026. URL: <https://github.com/ecrum19/VCF-RDFizer/tree/be658a2>, tag v3.0.3; converted and validated 14 September 2026.
- [33] A. Dimou, M. Vander Sande, P. Colpaert, R. Verborgh, E. Mannens, R. Van de Walle, RML: A generic language for integrated RDF mappings of heterogeneous data, in: *Proceedings of the 7th Workshop on Linked Data on the Web*, volume 1184 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2014. URL: https://ceur-ws.org/Vol-1184/ldow2014_paper_01.pdf.
- [34] World Wide Web Consortium, Shapes Constraint Language (SHACL), W3C Recommendation, World Wide Web Consortium, 2017. URL: <https://www.w3.org/TR/2017/REC-shacl-20170720/>.

- [35] E. Crum, R. Taelman, B. Buelens, G. Ertaylan, R. Verborgh, Semantifying genomic variant data: VCF to RDF conversion framework, in: *Proceedings of the 16th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, volume 4196 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025, pp. 199–200. URL: https://ceur-ws.org/Vol-4196/paper_6.pdf.
- [36] Anthropic, Claude Opus 5 (Extra) [Large language model], Accessed through the Claude Code desktop application, 2026. URL: <https://www.anthropic.com/claude>, used September 2026.
- [37] OpenAI, GPT-6 Astra (X-High) [Large language model], Accessed through the Codex desktop application, 2026. URL: <https://openai.com/>, used September 2026.
- [38] E. Crum, Ontology Companion Generator (OCG), Version 1.5.0. Open-source software, 2026. URL: https://github.com/ecrum19/ontology_companion_generator/tree/v1.5.0, MIT licence; released 18 September 2026. Documentation at https://ecrum19.github.io/ontology_companion_generator/.
- [39] The 1000 Genomes Project Consortium, A global reference for human genetic variation, *Nature* 526 (2015) 68–74. doi:10.1038/nature15393.
- [40] M. Grüninger, M. S. Fox, Methodology for the design and evaluation of ontologies, in: *Proceedings of the Workshop on Basic Ontological Issues in Knowledge Sharing, IJCAI-95*, Montreal, Canada, 1995. URL: <https://www.eil.utoronto.ca/wp-content/uploads/enterprise-modelling/papers/gruninger-ijcai95.pdf>.
- [41] S. Peroni, A simplified agile methodology for ontology development, in: *OWL: Experiences and Directions – Reasoner Evaluation*, volume 10161 of *Lecture Notes in Computer Science*, Springer, Cham, 2017, pp. 55–69. doi:10.1007/978-3-319-54627-8_5.
- [42] A. Gómez-Pérez, Evaluation of ontologies, *International Journal of Intelligent Systems* 16 (2001) 391–409. doi:10.1002/1098-111X(200103)16:3<391::AID-INT1014>3.0.CO;2-2.
- [43] S. Bishop, M. Fairbairn, M. Norrish, P. Sewell, M. Smith, K. Wansbrough, Rigorous specification and conformance testing techniques for network protocols, as applied to TCP, UDP, and sockets, in: *Proceedings of the 2005 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications (SIGCOMM)*, ACM, 2005, pp. 265–276. doi:10.1145/1080091.1080123.
- [44] Y. N. Niu, E. G. Roberts, D. Denisko, M. M. Hoffman, Assessing and assuring interoperability of a genomics file format, *Bioinformatics* 38 (2022) 3327–3336. doi:10.1093/bioinformatics/btac327.
- [45] M. Sabou, M. Fernandez, Ontology (network) evaluation, in: M. C. Suárez-Figueroa, A. Gómez-Pérez, E. Motta, A. Gangemi (Eds.), *Ontology Engineering in a Networked World*, Springer, Berlin, Heidelberg, 2012, pp. 193–212. doi:10.1007/978-3-642-24794-1_9.
- [46] M. C. Suárez-Figueroa, A. Gómez-Pérez, B. Villazón-Terrazas, How to write and use the ontology requirements specification document, in: *On the Move to Meaningful Internet Systems: OTM 2009*, volume 5871 of *Lecture Notes in Computer Science*, Springer, Berlin, Heidelberg, 2009, pp. 966–982. doi:10.1007/978-3-642-05151-7_16.
- [47] M. Poveda-Villalón, A. Fernández-Izquierdo, M. Fernández-López, R. García-Castro, LOT: An industrial oriented ontology engineering framework, *Engineering Applications of Artificial Intelligence* 111 (2022) 104755. doi:10.1016/j.engappai.2022.104755.
- [48] World Wide Web Consortium, SPARQL 1.1 Query Language, W3C Recommendation, World Wide Web Consortium, 2013. URL: <https://www.w3.org/TR/2013/REC-sparql11-query-20130321/>.
- [49] E. Blomqvist, A. Seil Sepour, V. Presutti, Ontology testing: Methodology and tool, in: *Knowledge Engineering and Knowledge Management*, volume 7603 of *Lecture Notes in Computer Science*, Springer, Berlin, Heidelberg, 2012, pp. 216–226. doi:10.1007/978-3-642-33876-2_20.
- [50] D. Wiśniewski, J. Potoniec, A. Ławrynowicz, C. M. Keet, Analysis of ontology competency questions and their formalizations in SPARQL-OWL, *Journal of Web Semantics* 59 (2019) 100534. doi:10.1016/j.websem.2019.100534.

- [51] G. Haesendonck, W. Maroy, P. Heyvaert, R. Verborgh, A. Dimou, Parallel RDF generation from heterogeneous big data, in: *Proceedings of the International Workshop on Semantic Big Data (SBD '19)*, Association for Computing Machinery, New York, NY, USA, 2019, pp. 1–6. doi:10.1145/3323878.3325802.
- [52] N. Matentzoglou, J. P. Ballhoff, S. M. Bello, C. Bizon, M. Brush, T. J. Callahan, C. G. Chute, W. D. Duncan, C. T. Evelo, D. Gabriel, J. Graybeal, A. Gray, B. M. Gyori, M. Haendel, H. Harmse, N. L. Harris, I. Harrow, H. B. Hegde, A. L. Hoyt, C. T. Hoyt, D. Jiao, E. Jiménez-Ruiz, S. Jupp, H. Kim, S. Koehler, T. Liener, Q. Long, J. Malone, J. A. McLaughlin, J. A. McMurry, S. Moxon, M. C. Munoz-Torres, D. Osumi-Sutherland, J. A. Overton, B. Peters, T. Putman, N. Queralt-Rosinach, K. Shefchek, H. Solbrig, A. Thessen, T. Tudorache, N. Vasilevsky, A. H. Wagner, C. J. Mungall, A simple standard for sharing ontological mappings (SSSOM), *Database* 2022 (2022) baac035. URL: <https://academic.oup.com/database/article/doi/10.1093/database/baac035/6591806>. doi:10.1093/database/baac035.
- [53] World Wide Web Consortium, SKOS Simple Knowledge Organization System Reference, W3C Recommendation, World Wide Web Consortium, 2009. URL: <https://www.w3.org/TR/skos-reference/>.
- [54] S. Beier, A. Fiebig, C. Pommier, I. Liyanage, M. Lange, P. J. Kersey, S. Weise, R. Finkers, B. Koylass, T. Cezard, M. Courtot, B. Contreras-Moreira, G. Naamati, S. Dyer, U. Scholz, Recommendations for the formatting of variant call format (VCF) files to make plant genotyping data FAIR, *F1000Research* 11 (2022) 231. doi:10.12688/f1000research.109080.2, version 2; peer review: 2 approved.
- [55] Med2RDF contributors, Med2RDF ontology, GitHub repository, n.d. URL: <https://github.com/med2rdf/med2rdf-ontology>, accessed 8 September 2026.
- [56] gvar, Genomic variants schema (gvar), GitHub repository, n.d. URL: <https://github.com/swat4hcls-2025-genomic-variation/genomic-variant-schema>, accessed 6 September 2026.
- [57] C. R. Lee, J. A. Luzum, K. Sangkuhl, R. S. Gammal, M. S. Sabatine, C. M. Stein, D. F. Kisor, N. A. Limdi, Y. M. Lee, S. A. Scott, J.-S. Hulot, D. M. Roden, A. Gaedigk, K. E. Caudle, T. E. Klein, J. A. Johnson, A. R. Shuldiner, Clinical Pharmacogenetics Implementation Consortium guideline for CYP2C19 genotype and clopidogrel therapy: 2022 update, *Clinical Pharmacology & Therapeutics* 112 (2022) 959–967. doi:10.1002/cpt.2526.
- [58] PharmCAT contributors, PharmCAT methods, Official documentation, 2026. URL: <https://pharmcat.org/methods/>, accessed 9 September 2026.
- [59] Human Phenotype Ontology Consortium, The Human Phenotype Ontology, Project website and documentation, 2026. URL: <https://hpo.jax.org/>, accessed 9 September 2026.
- [60] M. Griffith, N. C. Spies, K. Krysiak, J. F. McMichael, A. C. Coffman, A. M. Danos, B. J. Ainscough, C. A. Ramirez, D. T. Rieke, L. Kujan, E. K. Barnell, A. H. Wagner, Z. L. Skidmore, A. Wollam, C. J. Liu, M. R. Jones, R. L. Bilski, R. Lesurf, Y.-Y. Feng, N. M. Shah, M. Bonakdar, L. Trani, M. Matlock, A. Ramu, K. M. Campbell, G. C. Spies, A. P. Graubert, K. Gangavarapu, J. M. Eldred, D. E. Larson, J. R. Walker, B. M. Good, C. Wu, A. I. Su, R. Dienstmann, A. A. Margolin, D. Tamborero, N. Lopez-Bigas, S. J. M. Jones, R. Bose, D. H. Spencer, L. D. Wartman, R. K. Wilson, E. R. Mardis, O. L. Griffith, CIViC is a community knowledgebase for expert crowdsourcing the clinical interpretation of variants in cancer, *Nature Genetics* 49 (2017) 170–174. doi:10.1038/ng.3774.
- [61] A. Andrejev, T. Risch, Scientific SPARQL: Semantic Web queries over scientific data, in: *2012 IEEE 28th International Conference on Data Engineering Workshops (ICDEW)*, IEEE, 2012, pp. 5–10. doi:10.1109/ICDEW.2012.67.